



PharmaCare PMS

Frontend Documentation

A complete technical reference for the Next.js-based multi-tenant Pharmacy Management System frontend.

Next.js 16 · App Router

React 19

TypeScript 5

Tailwind CSS v4

Multi-tenant SaaS

Project: pms-frontend · Version: 1.0.0 · Generated: August 2026

Table of Contents

13 sections · Complete technical reference

01	Project Overview	
02	Tech Stack	
03	Project Structure	
04	Environment Configuration	
05	Authentication & Authorization	
06	Subscription System	
07	Application Routes (Pages)	
08	Services Layer (API Calls)	
09	Data Types (TypeScript Interfaces)	
10	Components	
11	State Management	
12	API Client	
13	Getting Started	

PharmaCare PMS is a multi-tenant SaaS Pharmacy Management System frontend built with Next.js. It connects to a .NET backend REST API and provides a full-featured dashboard for pharmacies to manage their day-to-day operations.

The system covers the following functional areas:

- Medicine inventory with batch tracking and expiry alerts
- Sales / Point-of-Sale (POS) and full sales history with printable invoices
- Customer, supplier, manufacturer, and category management
- Units of measure (UoM) per medicine with conversion factors
- Disposal records for expired or damaged medicines
- Reporting and analytics — daily/monthly sales, top medicines, low stock, stock consumption
- Role-based access control — **PharmacyAdmin** and **Pharmacist**
- Subscription management — Trial, Silver, Gold, and Diamond plans

The public landing page (/) showcases the product to prospective customers. All pharmacy management pages are behind authentication. The system is fully multi-tenant: each pharmacy is an isolated tenant identified by a UUID `tenantId` embedded in the JWT.

LAYER	TECHNOLOGY	VERSION
Framework	Next.js (App Router)	16.2.12
UI Library	React	19.2.4
Language	TypeScript	^5
Styling	Tailwind CSS + tailwind-merge + clsx	v4
HTTP Client	Axios	^1.7.9
Forms	react-hook-form + zod + @hookform/resolvers	^7.54 / ^3.24
Icons	lucide-react	^0.468.0
Date Utilities	date-fns	^4.1.0
Notifications	react-hot-toast	^2.4.1
Font	Geist (via next/font)	—
Linting	ESLint + eslint-config-next	^9

```

pms-frontend/
├─ src/
│   ├─ app/                                # Next.js App Router pages
│   │   ├─ page.tsx                        # Landing / marketing page
│   │   ├─ layout.tsx                      # Root layout (AuthProvider, fonts)
│   │   ├─ globals.css                    # Global Tailwind styles
│   │   ├─ login/page.tsx                 # Login page
│   │   ├─ signup/page.tsx                # Tenant & pharmacist signup
│   │   ├─ dashboard/page.tsx             # Main dashboard
│   │   ├─ medicines/page.tsx             # Medicine management
│   │   ├─ batches/page.tsx               # Batch management
│   │   ├─ units/page.tsx                 # Units of Measure management
│   │   ├─ sales/page.tsx                 # Point-of-Sale (create sale)
│   │   ├─ sales-history/page.tsx         # View past sales
│   │   ├─ customers/page.tsx             # Customer management
│   │   ├─ suppliers/page.tsx             # Supplier management
│   │   ├─ manufacturers/page.tsx         # Manufacturer management
│   │   ├─ categories/page.tsx            # Category management
│   │   ├─ disposals/page.tsx             # Disposal records
│   │   ├─ near-expiry/page.tsx           # Near-expiry batch alerts
│   │   ├─ reports/page.tsx               # Analytics & reports
│   │   ├─ features/page.tsx              # Public features/pricing page
│   │   └─ subscription/
│   │       ├─ billing/                   # Subscription billing page
│   │       └─ expired/                   # Subscription expired page
│   └─ components/
│       ├─ layout/
│       │   ├─ AppLayout.tsx              # Authenticated page wrapper
│       │   ├─ Sidebar.tsx                # Navigation sidebar
│       │   └─ Navbar.tsx                 # Top navbar
│       └─ ui/
│           ├─ Badge.tsx
│           ├─ BillPrint.tsx              # Invoice/bill print component
│           ├─ ConfirmDialog.tsx
│           ├─ FormField.tsx
│           ├─ LoadingSpinner.tsx
│           ├─ MedicineRow.tsx            # POS medicine row component
│           ├─ Modal.tsx
│           ├─ StatCard.tsx               # Dashboard KPI stat card
│           └─ UnitsManager.tsx
│   └─ context/
│       └─ AuthContext.tsx                # Auth state + subscription state

```

```

|   |─ lib/
|   |   |─ api.ts           # Axios instance + interceptors
|   |   └─ utils.ts        # cn() utility helper
|   └─ services/           # API service modules (one per domain)
|       |─ authService.ts
|       |─ batchService.ts
|       |─ categoryService.ts
|       |─ customerService.ts
|       |─ disposalService.ts
|       |─ manufacturerService.ts
|       |─ medicineService.ts
|       |─ medicineUnitService.ts
|       |─ reportService.ts
|       |─ saleService.ts
|       |─ subscriptionService.ts
|       |─ supplierService.ts
|       └─ uomService.ts
|   └─ types/
|       └─ index.ts        # All TypeScript interfaces and DTOs
└─ public/                # Static assets (SVGs)
└─ .env.example           # Environment variable template
└─ next.config.ts         # Next.js config with env validation
└─ tsconfig.json
└─ package.json

```

4

Environment Configuration

Copy `.env.example` to `.env.local` before running the project. All variables prefixed with `NEXT_PUBLIC_` are exposed to the browser.

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
<code>NEXT_PUBLIC_API_URL</code>	YES	<code>http://localhost:5259</code>	Base URL of the .NET backend API (no trailing slash)
<code>NEXT_PUBLIC_APP_NAME</code>	NO	<code>PharmaCare PMS</code>	Display name shown in the sidebar and browser title
<code>NEXT_PUBLIC_APP_VERSION</code>	NO	<code>1.0.0</code>	Version string shown below the app name in the sidebar

 **Build-time validation:** `NEXT_PUBLIC_API_URL` is checked in `next.config.ts` at startup. If it is missing, the build/start command will throw an error and refuse to continue.

Login Flow

- 1 User visits `/login` and submits email + password.
- 2 `authService.login()` calls `POST /api/auth/login` on the backend.
- 3 On success, the JWT token and user object are stored in `localStorage` under the keys `pms_token` and `pms_user`.
- 4 Subscription data is fetched immediately (`GET /api/subscription/mine`) and stored as `pms_subscription`.
- 5 All subsequent API calls attach the token via Axios request interceptor as `Authorization: Bearer <token>`.

Auth Context API

`AuthContext` (`src/context/AuthContext.tsx`) is the single source of truth for auth state. It is provided at the root layout and can be consumed with `useAuth()`.

VALUE / METHOD	TYPE	DESCRIPTION
<code>user</code>	<code>AuthUser null</code>	Logged-in user (email, fullName, roles, userId, tenantId)
<code>subscription</code>	<code>SubscriptionInfo null</code>	Current tenant's subscription details
<code>loading</code>	<code>boolean</code>	True while hydrating from localStorage on mount
<code>isAuthenticated</code>	<code>boolean</code>	True when <code>user</code> is not null
<code>login(dto)</code>	<code>async fn</code>	Logs in and populates context; returns error message or null
<code>logout()</code>	<code>fn</code>	Clears localStorage and redirects to <code>/login</code>
<code>refreshSubscription()</code>	<code>async fn</code>	Re-fetches subscription from API and updates context

Axios Interceptors

HTTP STATUS	ACTION
401 UNAUTHORIZED	Clears <code>pms_token</code> , <code>pms_user</code> , <code>pms_subscription</code> from <code>localStorage</code> and redirects to <code>/login</code> .
402 PAYMENT REQUIRED	Redirects to <code>/subscription/expired</code> . Token is preserved so the user can access the billing page.

Roles

- **PharmacyAdmin** — Full access: can add pharmacists, manage subscriptions, and perform all operations.
- **Pharmacist** — Standard staff access for day-to-day pharmacy operations.

The application is multi-tenant and subscription-gated. Each pharmacy tenant must hold an active subscription to use the protected API endpoints.

Subscription Plans

Trial Max 20 medicines Time-limited Free tier	Silver Unlimited medicines Time-limited Paid tier	Gold Unlimited medicines Time-limited Paid tier	Diamond Unlimited medicines Lifetime access Premium tier
---	---	---	--

Subscription Status

STATUS	MEANING	API BEHAVIOUR
ACTIVE	Subscription is running normally.	All protected endpoints accessible.
EXPIRED	Subscription end date has passed.	Backend returns HTTP 402. App redirects to <code>/subscription/expired</code> .
CANCELLED	Subscription was cancelled by admin.	Treated same as Expired.

Sidebar Status Banner

The sidebar shows a live subscription status banner below the navigation links:

CONDITION	COLOUR	LABEL EXAMPLE
Expired	RED	"Subscription expired"
≤ 14 days remaining	ORANGE	"Gold — 8d left"
Diamond (Lifetime)	INDIGO	"Diamond — Lifetime"
Active	GREEN	"Silver — 45d left"

7 Application Routes (Pages)

Public Routes

ROUTE	DESCRIPTION
/	Landing / marketing page with feature highlights and plan overview
/features	Detailed features & pricing page
/login	Email + password login form
/signup	New pharmacy (tenant) self-registration form

Protected Routes (require valid JWT)

ROUTE	DESCRIPTION
<code>/dashboard</code>	KPIs, low stock alerts, expiring batches, and 6-month sales bar chart
<code>/medicines</code>	Full medicine CRUD — bulk import, unit management per medicine
<code>/batches</code>	Batch management — receive new stock, per-batch expiry tracking, bulk import
<code>/units</code>	Units of Measure management with conversion factors
<code>/sales</code>	Point-of-Sale — build a cart and generate an invoice
<code>/sales-history</code>	Browse past sales; view and print individual invoices
<code>/customers</code>	Customer CRUD (name, phone, address)
<code>/suppliers</code>	Supplier CRUD (name, phone, address)
<code>/manufacturers</code>	Manufacturer CRUD
<code>/categories</code>	Medicine category CRUD
<code>/disposals</code>	Record medicine disposals (expired, damaged stock)
<code>/near-expiry</code>	List batches expiring within a configurable day threshold
<code>/reports</code>	Top medicines, top suppliers/manufacturers, low stock, stock consumption
<code>/signup</code>	Add a new pharmacist to the current tenant (Admin only)
<code>/subscription/billing</code>	View current plan, renew or upgrade subscription
<code>/subscription/expired</code>	Locked page shown when HTTP 402 is returned

All HTTP calls go through `src/lib/api.ts` (the Axios instance). Each business domain has its own service module in `src/services/`.

authService

METHOD	ENDPOINT	DESCRIPTION
<code>login(dto)</code>	<code>POST /api/auth/login</code>	Authenticate user; returns JWT
<code>tenantSignup(dto)</code>	<code>POST /api/auth/tenant-signup</code>	Create new pharmacy (tenant) + first admin
<code>signup(dto)</code>	<code>POST /api/auth/signup</code>	Add pharmacist to existing tenant

subscriptionService

METHOD	ENDPOINT	DESCRIPTION
<code>getMine()</code>	<code>GET /api/subscription/mine</code>	Get current tenant's subscription
<code>renew(dto?)</code>	<code>POST /api/subscription/renew</code>	Renew with same plan
<code>upgrade(dto)</code>	<code>POST /api/subscription/upgrade</code>	Upgrade to a higher plan

medicineService

METHOD	ENDPOINT	DESCRIPTION
<code>getAll()</code>	<code>GET /api/medicine</code>	List all medicines
<code>create(dto)</code>	<code>POST /api/medicine</code>	Create a medicine
<code>update(id, dto)</code>	<code>PUT /api/medicine/{id}</code>	Update a medicine
<code>delete(id)</code>	<code>DELETE /api/medicine/{id}</code>	Delete a medicine
<code>bulkCreate(items)</code>	<code>POST /api/medicine/bulk</code>	Bulk import medicines

saleService

METHOD	ENDPOINT	DESCRIPTION
<code>create(dto)</code>	<code>POST /api/sales</code>	Create a new sale/invoice
<code>getAll()</code>	<code>GET /api/sales</code>	List all sales
<code>getById(id)</code>	<code>GET /api/sales/{id}</code>	Get a single sale with items

reportService

METHOD	ENDPOINT	DESCRIPTION
<code>getDashboard()</code>	<code>GET /api/report/dashboard</code>	Combined dashboard report
<code>getToday()</code>	<code>GET /api/report/today</code>	Today's sales summary
<code>getThisMonth()</code>	<code>GET /api/report/this-month</code>	This month's sales summary
<code>getMonthlySales(n)</code>	<code>GET /api/report/monthly-sales?months={n}</code>	Last n months of sales
<code>getLowStock()</code>	<code>GET /api/report/low-stock</code>	Medicines at/below reorder level
<code>getStockConsumption(p)</code>	<code>GET /api/report/stock-consumption</code>	Stock consumption with date/supplier/manufacture filters
<code>getTopMedicines(n)</code>	<code>GET /api/report/top-medicines?top={n}</code>	Top-selling medicines
<code>getTopManufacturers(n)</code>	<code>GET /api/report/top-manufacturers?top={n}</code>	Top manufacturers by revenue
<code>getTopSuppliers(n)</code>	<code>GET /api/report/top-suppliers?top={n}</code>	Top suppliers by volume

Other Services

SERVICE	BASE ENDPOINT	OPERATIONS
<code>batchService</code>	<code>/api/batch</code>	getAll, create, bulkCreate
<code>disposalService</code>	<code>/api/disposal</code>	getAll, getByld, create
<code>categoryService</code>	<code>/api/category</code>	Full CRUD
<code>customerService</code>	<code>/api/customer</code>	Full CRUD
<code>supplierService</code>	<code>/api/supplier</code>	Full CRUD
<code>manufacturerService</code>	<code>/api/manufacturer</code>	Full CRUD
<code>medicineUnitService</code>	<code>/api/medicine/{id}/units</code>	Full CRUD per medicine
<code>uomService</code>	<code>/api/uom</code>	Full CRUD

All interfaces and DTOs are centralized in `src/types/index.ts`. Key types are shown below.

Auth

```
interface LoginDto { email: string; password: string; }

interface TenantSignupDto {
  fullName: string; email: string;
  password: string; confirmPassword: string;
  pharmacyName: string;
}

interface AuthResponse {
  isSuccess: boolean; message: string;
  token: string | null;
  email: string | null; fullName: string | null;
  roles: string[];
  userId: number | null;
  tenantId: string | null; // UUID of the pharmacy tenant
}
```

Subscription

```
type PlanType = 'Trial' | 'Silver' | 'Gold' | 'Diamond';
type SubscriptionStatus = 'Active' | 'Expired' | 'Cancelled';

interface SubscriptionInfo {
  subscriptionId: number; tenantId: string;
  planType: PlanType; status: SubscriptionStatus;
  startDate: string; endDate: string | null; // null = Lifetime (Diamond)
  isActive: boolean;
  daysRemaining: number | null; // null = Lifetime or already expired
  medicineLimit: number | null; // null = unlimited; 20 for Trial
}
```


Medicine & Batches

```
interface Medicine {
  medicineId: number; name: string; genericName?: string | null;
  categoryId?: number | null; manufacturerId?: number | null;
  tabletsPerStrip: number; strapsPerBox: number;
  stripPrice: number; tabletPrice: number;
  reorderLevel: number; requiresPrescription: boolean;
  isActive: boolean; totalStockInTablets: number;
  units: MedicineUnit[];
}

interface MedicineBatch {
  batchId: number; medicineId: number; medicineName: string;
  batchNumber: string; supplierId?: number | null;
  expiryDate: string; quantityInTablets: number;
  purchasePricePerTablet: number; isExpired: boolean;
}
```

Sales / POS

```
type PaymentMode = 'Cash' | 'Card' | 'Online' | 'Credit';

interface CreateSaleRequestDto {
  customerId: number; pharmacistId: number;
  discountPercent: number; paymentMode: PaymentMode;
  items: CreateSaleItemDto[];
}

interface CartItem {
  id: string; medicineId: number; medicineName: string;
  medicineUnitId: number; uomName: string;
  quantity: number; discountPercent: number;
  unitPrice: number; availableUnits: MedicineUnitForPos[];
}
```

```
interface SalesSummary {
  totalSales: number; totalRevenue: number; totalItemsSold: number;
}

interface LowStockItem {
  medicineId: number; medicineName: string;
  totalStockInTablets: number; reorderLevel: number;
  shortfallInTablets: number;
}

interface StockConsumptionReport {
  fromDate?: string | null; toDate?: string | null;
  items: StockConsumptionItem[];
  totalQuantitySold: number; totalRevenue: number;
}
```

10 Components

Layout Components

COMPONENT	FILE	DESCRIPTION
AppLayout	<code>layout/AppLayout.tsx</code>	Wraps every authenticated page. Renders Sidebar + Navbar + scrollable content area. Accepts a <code>title</code> prop.
Sidebar	<code>layout/Sidebar.tsx</code>	Fixed left nav with 15 links, subscription status banner, user avatar + plan badge, and Sign Out button. Mobile-responsive with overlay.
Navbar	<code>layout/Navbar.tsx</code>	Top bar with hamburger menu (mobile) and current page title.

UI Components

COMPONENT	PURPOSE
StatCard	Dashboard KPI card — icon, value, title, subtitle, and colour variant (blue, emerald, purple, red, orange).
Badge	Inline status pill. Variants: <code>green</code> , <code>red</code> , <code>yellow</code> , <code>blue</code> , <code>gray</code> .
Modal	Accessible overlay modal with backdrop. Used by forms and dialogs.
ConfirmDialog	Reusable delete/action confirmation prompt built on top of Modal.
FormField	Labeled input wrapper with error message display; integrates with react-hook-form.
LoadingSpinner	Inline spinning indicator. <code>PageLoader</code> variant fills the full content area.
MedicineRow	POS cart row — medicine search/select, unit dropdown, quantity input, and per-item discount field.
BillPrint	Printable invoice layout rendered when a sale is completed. Displays itemised receipt.
UnitsManager	Embedded CRUD UI for managing medicine-specific units of measure with conversion factors.

11 State Management

The project uses **React Context** for global state only. There is no Redux, Zustand, or other external store.

LAYER	MECHANISM	WHAT IT HOLDS
Global auth & subscription	<code>AuthContext</code> (React Context)	Logged-in user, subscription info, login/logout actions
Page / feature state	<code>useState</code> + <code>useEffect</code>	Lists, form data, loading and error states per page
Forms	react-hook-form + zod	Field values, validation errors, submission state
Persistence	localStorage	JWT token, user object, subscription info across page reloads

`src/lib/api.ts` exports a pre-configured Axios instance that all service modules import and use.

```
// src/lib/api.ts
const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL, // e.g. http://localhost:5259
  headers: { 'Content-Type': 'application/json' },
});

// Attach JWT on every request
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('pms_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Handle auth and payment errors globally
api.interceptors.response.use(
  (res) => res,
  (error) => {
    const status = error.response?.status;
    if (status === 401) {
      localStorage.removeItem('pms_token');
      localStorage.removeItem('pms_user');
      localStorage.removeItem('pms_subscription');
      window.location.href = '/login';
    }
    if (status === 402) {
      window.location.href = '/subscription/expired';
      // Note: token is NOT cleared – user needs it to access billing page
    }
    return Promise.reject(error);
  }
);
```

13 Getting Started

Prerequisites

- Node.js 18 or later
- npm (bundled with Node.js)
- The .NET PMS backend API running (default: `http://localhost:5259`)

Setup & Run

```
# 1. Enter the project directory
cd pms-frontend

# 2. Install dependencies
npm install

# 3. Configure environment variables
cp .env.example .env.local
# Open .env.local and set NEXT_PUBLIC_API_URL to your backend URL

# 4. Start the development server
npm run dev
```

Open `http://localhost:3000` in your browser.

Available Scripts

COMMAND	DESCRIPTION
<code>npm run dev</code>	Start development server with Turbopack (hot reload)
<code>npm run build</code>	Create an optimised production build
<code>npm start</code>	Serve the production build
<code>npm run lint</code>	Run ESLint across the codebase

First-time Onboarding

- 1 Go to the landing page (/) and click **Get started free**.

- 2 Fill in the signup form: your full name, email, password, and pharmacy name. This creates a brand-new tenant + PharmacyAdmin account.
 - 3 A **Trial** subscription is assigned automatically (max 20 medicines).
 - 4 Log in at `/login` and land on the Dashboard.
 - 5 Navigate to `/categories` , `/manufacturers` , and `/suppliers` to seed reference data, then add medicines via `/medicines` .
 - 6 Receive stock batches at `/batches` , add customers at `/customers` , then start selling at `/sales` .
 - 7 When ready, upgrade your subscription at `/subscription/billing` .
-